Operating Systems Lab

Class Assignment — Section A2

Least-Usage Fair Scheduling

30 minutes. Fresh xv6 checkout, `CPUS := 1`.

The Idea

Replace the round-robin scheduler with one that always runs whoever has had the **least** CPU time so far, weighted by **priority**.

- Every process starts with usage score `0` and priority `DEFAULT_PRIORITY`.
- To pick who runs next, choose the *runnable* process with the smallest usage score (ties broken by table order).
- Every time a process is picked and runs, add `(MAX_PRIORITY + 1 - priority)` to its usage score — a priority-10 process's score barely moves; a priority-1 process's jumps a lot.
- There is no fixed turn length or queue: the "smallest score wins" rule alone decides everything, on every single reschedule.

Specification Parameters

Macro	Value	Meaning
`MIN_PRIORITY`	1	Lowest priority a process may request.
`MAX_PRIORITY`	10	Highest priority a process may request.
`DEFAULT_PRIORITY`	5	Priority every process starts with.

System Calls

```
int setpriority(int priority);     int getpriority(void);
```

`setpriority` sets the calling process's priority; returns `0`, or `-1` (leaving/resetting the priority to `DEFAULT_PRIORITY`) if `priority` is outside `[MIN_PRIORITY, MAX_PRIORITY]`. `getpriority` returns the current value. A `fork()` child inherits its parent's *current* priority, but starts with usage score `0`.

Logging

Toggleable log line, printed every time the scheduler picks a process to run:

```
PICK: Process 5 (testloop) selected with usage 42 at priority 4
```

Test Program

Use the attached `testloop.c` as-is: save it to `user/testloop.c` and add `$U/_testloop` to `UPROGS` in the Makefile. Calling syntax: `testloop <iterations> <priority>`.

Sample Session

Three testloop processes, equal work, different priorities, launched one line at a time:

```
$ testloop 60 8 &
PID 4: starting 60 iterations at time 42, priority 8
$ testloop 60 4 &
PID 6: starting 60 iterations at time 60, priority 4
$ testloop 60 2 &
PID 8: starting 60 iterations at time 74, priority 2
PID 4: finished at time 122
PID 6: finished at time 164
PID 8: finished at time 175
```

Each pick raises process 4's (priority 8) usage by only 3, process 6's (priority 4) by 7, process 8's (priority 2) by 9. Equal work, but priority 8 finishes first (122), well ahead of priority 4 (164) and priority 2 (175).

Submission: `git diff HEAD > ../<studentID>.patch`